

TUTORIAL – 3

Requirement Reviews & Inspections

Understand Requirement Reviews, Code Walkthroughs, Inspections, and Peer Review Activities Using Sample Project Documents.

Aim

To understand Requirement Reviews, Code Walkthroughs, Inspections, and Peer Review activities using sample project documents to identify software defects at an early stage of the Software Development Life Cycle (SDLC).

Description

Static Testing is a software quality assurance technique used to detect defects in software artifacts without executing the application. Unlike Dynamic Testing, Static Testing focuses on reviewing requirement specifications, design documents, source code, and other project-related documents to identify ambiguities, inconsistencies, missing requirements, logical errors, coding standard violations, and documentation defects during the early phases of software development.

The primary static testing techniques include Requirement Reviews, Walkthroughs, Inspections, and Peer Reviews. Requirement Reviews verify that requirements are complete, correct, consistent, and testable. Code Walkthroughs involve developers explaining the source code to reviewers to identify logical and implementation issues. Inspections are formal review processes conducted using predefined checklists to detect defects systematically. Peer Reviews are collaborative evaluations performed by team members to improve software quality and knowledge sharing.

In this practical, an **Online Banking System Login Module** is considered as the sample project. The requirement document and source code are reviewed to identify missing validations, security issues, requirement ambiguities, coding defects, and improvement opportunities before executing the software.

Problem Statement

Perform Requirement Review, Code Walkthrough, Inspection, and Peer Review on an **Online Banking System Login Module** to identify requirement defects, coding issues, security vulnerabilities, missing validations, and documentation inconsistencies before software execution using Static Testing techniques.

Implementation Details

Parameter	Details
Project Name	Online Banking System
Application/Software	Banking Management Application
Module Name	Login Authentication Module
SDLC Model	Waterfall Model
Programming Language	Python 3.x
IDE/Tool Used	Visual Studio Code / PyCharm / IDLE
Input/Test Data	Username = admin, Password = bank123

Source Code

```
# -----  
# Tutorial 3  
# Requirement Reviews & Inspections  
# Sample Project: Online Banking Login Module  
# -----  
  
# Store valid credentials  
  
valid_username = "admin"  
valid_password = "bank123"  
  
# Accept input  
  
username = input("Enter Username : ")  
password = input("Enter Password : ")  
  
# Validate blank fields  
  
if username == "" or password == "":  
    print("Username and Password cannot be empty")  
  
# Verify credentials  
  
elif username == valid_username and password == valid_password:  
    print("Login Successful")  
  
# Invalid credentials
```

else:

```
print("Invalid Username or Password")
```

Test Case Detailed

Field	Details
Test Case ID	TC_LOGIN_001
Requirement ID	RQ_LOGIN_001
Module Name	Login Module
Feature Under Test	User Authentication
Test Scenario	Verify login using valid username and password
Test Objective	Validate successful login functionality
Test (Unit/Integration/System/UAT) Type	Unit Testing
Test Technique (Black Box/White Box)	Black Box Testing
Priority	High
Severity	Critical
Preconditions	Application is installed and running
Postconditions	User successfully logs into the application
Test Environment	Windows 11, Python 3.12
Browser/OS (if applicable)	Not Applicable
Test Data	Username = admin Password = bank123
Expected Result	Login Successful
Actual Result	Login Successful
Status (Pass/Fail)	Pass
Executed By	Student
Execution Date	DD/MM/YYYY

Test Case Execution

Step No.	Test Steps	Input	Expected Result	Actual Result	Status
1	Launch the application	Start Program	Application starts successfully	Application started	Pass
2	Enter valid username	admin	Username accepted	Username accepted	Pass
3	Enter valid password	bank123	Password accepted	Password accepted	Pass
4	Click Login	Valid Credentials	Login Successful	Login Successful	Pass
5	Enter blank username	""	Validation message displayed	Validation message displayed	Pass
6	Enter blank password	""	Validation message displayed	Validation message displayed	Pass
7	Enter invalid password	test123	Invalid Username or Password	Invalid Username or Password	Pass
8	Close application	Exit	Program terminated	Program terminated	Pass

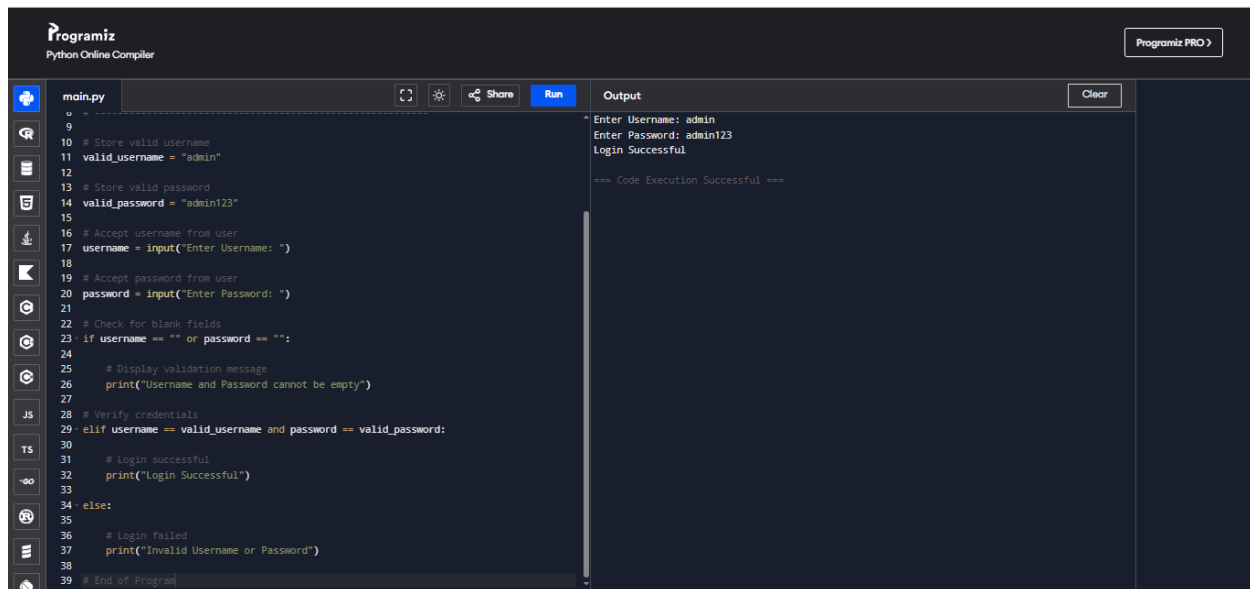
Testing Activities Performed

Sr. No.	SDLC Phase	STLC Phase	Testing Activity	Deliverable/Artifact
1	Requirement Analysis	Requirement Analysis	Requirement Review	Requirement Review Report
2	System Design	Test Planning	Design Review & Inspection	Inspection Checklist
3	Development	Test Design	Code Walkthrough & Peer Review	Code Review Report
4	Testing	Test Execution	Test Case Review & Defect Analysis	Test Cases & Defect Log
5	Deployment & Maintenance	Test Closure	Documentation Review & Change Request Analysis	Final Review Report

Observation

Sr. No.	Observation	Remarks
1	Requirement Reviews identified missing functional and security requirements.	Early defect detection reduces development cost.
2	Code Walkthrough detected logical and validation-related issues.	Improves source code quality.
3	Inspection ensured compliance with coding standards and documentation.	Formal defect detection process.
4	Peer Review improved collaboration and knowledge sharing among developers.	Better software maintainability.
5	Static Testing helped identify defects before software execution.	Improves overall software quality.

Output



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains a Python script for a login system. The script defines valid username and password, prompts the user for input, checks for empty fields, and verifies the credentials. The output window on the right shows the execution results: 'Enter Username: admin', 'Enter Password: admin123', 'Login Successful', and '=== Code Execution Successful ==='. The code is as follows:

```
9
10 # Store valid username
11 valid_username = "admin"
12
13 # Store valid password
14 valid_password = "admin123"
15
16 # Accept username from user
17 username = input("Enter Username: ")
18
19 # Accept password from user
20 password = input("Enter Password: ")
21
22 # Check for blank fields
23 if username == "" or password == "":
24     # Display validation message
25     print("Username and Password cannot be empty")
26
27
28 # Verify credentials
29 elif username == valid_username and password == valid_password:
30
31     # login successful
32     print("Login Successful")
33
34 else:
35
36     # login failed
37     print("Invalid Username or Password")
38
39 # End of Program
```

Output 1 (Valid Login)

Enter Username : admin
Enter Password : bank123

Login Successful

Output 2 (Blank Username)

Enter Username :

Enter Password : bank123

Username and Password cannot be empty

Output 3 (Invalid Password)

Enter Username : admin

Enter Password : test123

Invalid Username or Password

Result

The practical was successfully completed. Requirement Reviews, Code Walkthroughs, Inspections, and Peer Review activities were performed on the Online Banking System Login Module. Static testing techniques effectively identified requirement defects, coding issues, and missing validations before software execution, thereby improving software quality and reducing future maintenance effort.